

ITF23019: Parallel and Distributed Programming

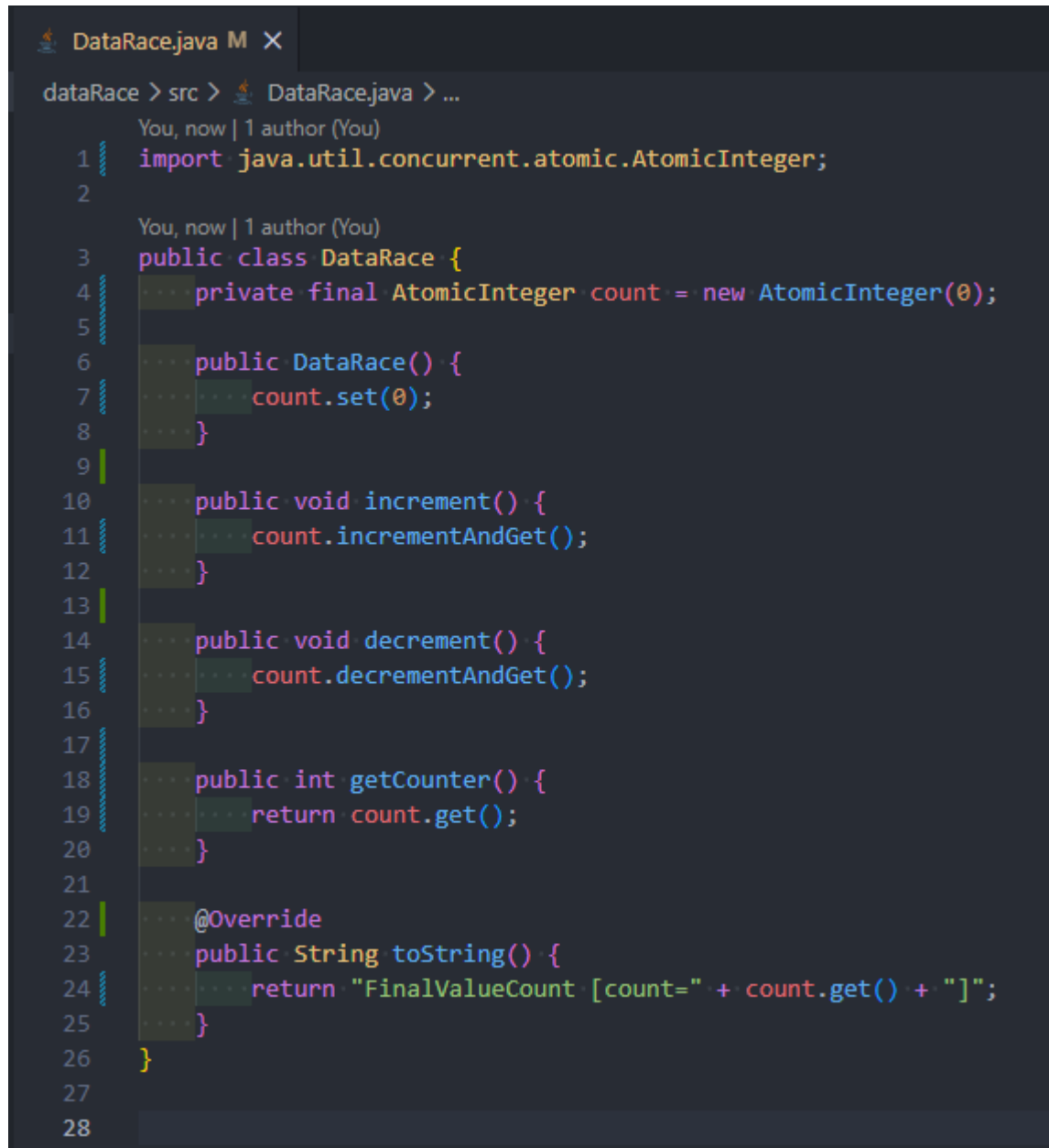
Report for Lab 8: Additional Thread-Safety Issues

Name: Emil Berglund

GitHub: EmilB04

Exercise 1: Fix data race problem

In this exercise I updated the code by replacing `count++` and `count--` with Atomic variables such as `incrementAndGet` and `decrementAndGet` to avoid data race problems. Below is a screenshot of the changed code. It can also be found in the `DataRace.java` file



```
dataRace > src > DataRace.java > ...
You, now | 1 author (You)
1  import java.util.concurrent.atomic.AtomicInteger;
2
You, now | 1 author (You)
3  public class DataRace {
4      private final AtomicInteger count = new AtomicInteger(0);
5
6      public DataRace() {
7          count.set(0);
8      }
9
10     public void increment() {
11         count.incrementAndGet();
12     }
13
14     public void decrement() {
15         count.decrementAndGet();
16     }
17
18     public int getCounter() {
19         return count.get();
20     }
21
22     @Override
23     public String toString() {
24         return "FinalValueCount [count=" + count.get() + "]";
25     }
26 }
27
28
```

Exercise 2: Run code and duration

Subtask 1: Run the code

```
Parallel-distributed-programming/lab8 on main [!?]
> cd /home/emilb_linux/dev/V2026/Parallel-distributed-programming/lab8
k-amd64/bin/java -XX:+ShowCodeDetailsInExceptionMessages -Xmx1024m -Xms128m -Xss1024k -Xlog:gc:/home/emilb_linux/dev/V2026/Parallel-distributed-programming/workspace/c314c783153e84638b8210cc860a7650/redhat.java/jdt_ws
Thread started...
Thread stopped.
Thread started...
Thread stopped.
Thread started...
Thread stopped.
Thread started...
Thread stopped.
Thread started...
Thread stopped.
Thread started...
Thread stopped.

Parallel-distributed-programming/lab8 on main [!?]
>
```

Subtask 2: Duration of run

By adding `System.currentTimeMillis()` before and after the for-loop to get end time – start time, I calculated the running time.

Total execution time: 1 ms

Exercise 3: Run code and explain

Subtask 1: Run code

Disclaimer: The method *processData* in *Operations.java* was defined with `TimeUnit` in seconds, and not milliseconds. This led to the program not running, after waiting for over a minute. I therefore changed it to milliseconds, which means that all methods in *Operations.java* now use milliseconds.

Under is the code running twice in a row.

```
Parallel-distributed-programming/lab8/criticalSection on main [!?]
● > cd /Users/emil/Documents/GitHub/V26/Parallel-distributed-programming/lab8
es/temurin-25.jdk/Contents/Home/bin/java --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -Xmx1024m -Xms128m -Xss1024k -Xlog:gc:/Users/emil/Documents/GitHub/V26/Parallel-distributed-programming/workspace/Code/User/workspaceStorage/98d6c5832c9f0636905b999c416a50d8/redhat.java
Main: First Approach: 920
Main: Second Approach: 589

Parallel-distributed-programming/lab8/criticalSection on main [!?]
● > cd /Users/emil/Documents/GitHub/V26/Parallel-distributed-programming/lab8
es/temurin-25.jdk/Contents/Home/bin/java --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -Xmx1024m -Xms128m -Xss1024k -Xlog:gc:/Users/emil/Documents/GitHub/V26/Parallel-distributed-programming/workspace/Code/User/workspaceStorage/98d6c5832c9f0636905b999c416a50d8/redhat.java
Main: First Approach: 923
Main: Second Approach: 576

Parallel-distributed-programming/lab8/criticalSection on main [!?]
○ > |
```

Subtask 2: Explain why execution times are different

Both tasks use a shared ReentrantLock across 10 threads, and each calls the same three operations: readData(): 30ms; processData(): 30ms; writeData(): 20ms

The difference lies in how much work is done inside the critical section.

Task1 acquires the lock once and holds it for all three operations. Task2 only locks around operations that truly require mutual exclusion, leaving readData() outside the critical section.

To summarize and answer. Both tasks share the same lock and call the same three operations. However, the critical section is different, and this is where their operations differ. Task2 is faster because it applies the principle of minimizing the critical section. This means that work that is independent or ready-only is moved outside the lock, so threads can execute it in parallel, reducing contention and improving throughput.

Exercise 4: Fix deadlock problem

The deadlock problem has been solved, and code is updated in the provided java-project. Both fixes boil down to one rule: always acquire locks in the same order.